Client-Server Architecture Case Study

Application: Wikipedia
Name: Abdul Hannan Khan **Roll No:** F24-127

1. Application and the Internet versus the Web

Wikipedia is a publicly accessible web application for reading, searching, and editing encyclopedia articles. This case study follows a reader opening https://en.wikipedia.org/wiki/Computer in a browser. The browser acts as the **client**: it requests information and displays the result. Wikimedia's servers provide the content and services needed to answer that request.

The **Internet** is the underlying network that carries data between the reader's device and Wikimedia's infrastructure. The **World Wide Web** is a service that uses this network to provide linked pages through browsers and HTTP. In this example, the Internet transports the data, while Wikipedia's pages, links, and web addresses belong to the Web. Other Internet services, such as email, can use the same network without being Wikipedia web pages. [1]

2. Why Wikipedia is a dynamic application

Wikipedia is **dynamic** because its content and responses depend on stored information and user actions. An article can change when an editor saves a revision; a search produces results for the entered words; and signing in allows account-specific features. These functions require application logic and persistent data. Wikipedia uses MediaWiki software to support them. Its pages are not simply a collection of fixed HTML files that a developer must manually replace after every edit. [2]

A page may still contain static resources, including images, stylesheets, and JavaScript files. Wikimedia also caches generated responses so many readers can receive the same saved result quickly. Caching does not make the whole application static: the content still comes from a system that processes edits and other requests. A cached article can be replaced when its underlying information changes. [2, 3]

3. HTTPS, ports, and hosting

The example URL uses **HTTPS**, which carries HTTP messages over an encrypted connection protected by TLS. Encryption helps protect article requests, login credentials, and session information while they travel across networks. Certificate validation helps the browser check that it is communicating with the intended website. Ordinary HTTP does not provide these protections. HTTPS protects the connection; it does not prove that an article's statements are accurate. [4, 5]

The standard destination **port** for HTTPS is **443**; HTTP normally uses port 80. A port identifies a service at a network address. The browser therefore uses port 443 even though the example URL does not display it. The hostname identifies the destination by name, while the path /wiki/Computer identifies the requested resource. The browser also uses a temporary local port for its side of the connection.

Hosting means running the application and keeping its content available on network-connected infrastructure. Wikimedia operates cache servers and core services across data centers. The domain name gives readers a convenient address, while the hosted servers actually handle requests. Wikipedia is therefore a distributed service, rather than one physical computer containing every component. [3]

4. From URL entry to the displayed page

1. Interpret the URL and resolve the name. The browser separates the HTTPS scheme, en.wikipedia.org hostname, and article path. Through **DNS resolution**, it obtains an IP address for the hostname. A cached DNS answer may already be available; otherwise a resolver looks up the answer. DNS locates the destination but does not retrieve the article or translate its full path. [1]

2. Establish a secure connection. The browser connects to the destination on port 443 and negotiates a secure connection. With HTTP/1.1 or HTTP/2 this uses TCP followed by TLS; HTTP/3 uses QUIC over UDP with TLS integrated. Certificate checks and encryption take place before protected application data is exchanged. An existing connection may be reused. [5]

3. Send the request. For the article, the browser sends an HTTP **GET** request containing the resource path and headers identifying the host and client preferences. Relevant cookies may accompany the request if present. Reading an article does not require a request body. HTTP follows a **request-response cycle**: the client asks for a resource and the server returns a result. [4]

4. Process the request. Wikimedia's caching layer may return a stored article response immediately. If the request needs application processing, it is forwarded to MediaWiki. The backend identifies the page and obtains the required content from caches or storage, querying databases when necessary. It then prepares a response. A fresh database query is therefore not required for every article view. [2, 3]

5. Return the response. A successful page response normally includes the **HTTP status code** 200, response headers, and an HTML body. Headers describe details such as the content type and caching rules. A redirect uses a 3xx status and a destination; an unavailable resource may produce 404. The status tells the client how the request was handled. These are illustrative outcomes, not recorded results from a browser inspection. [4]

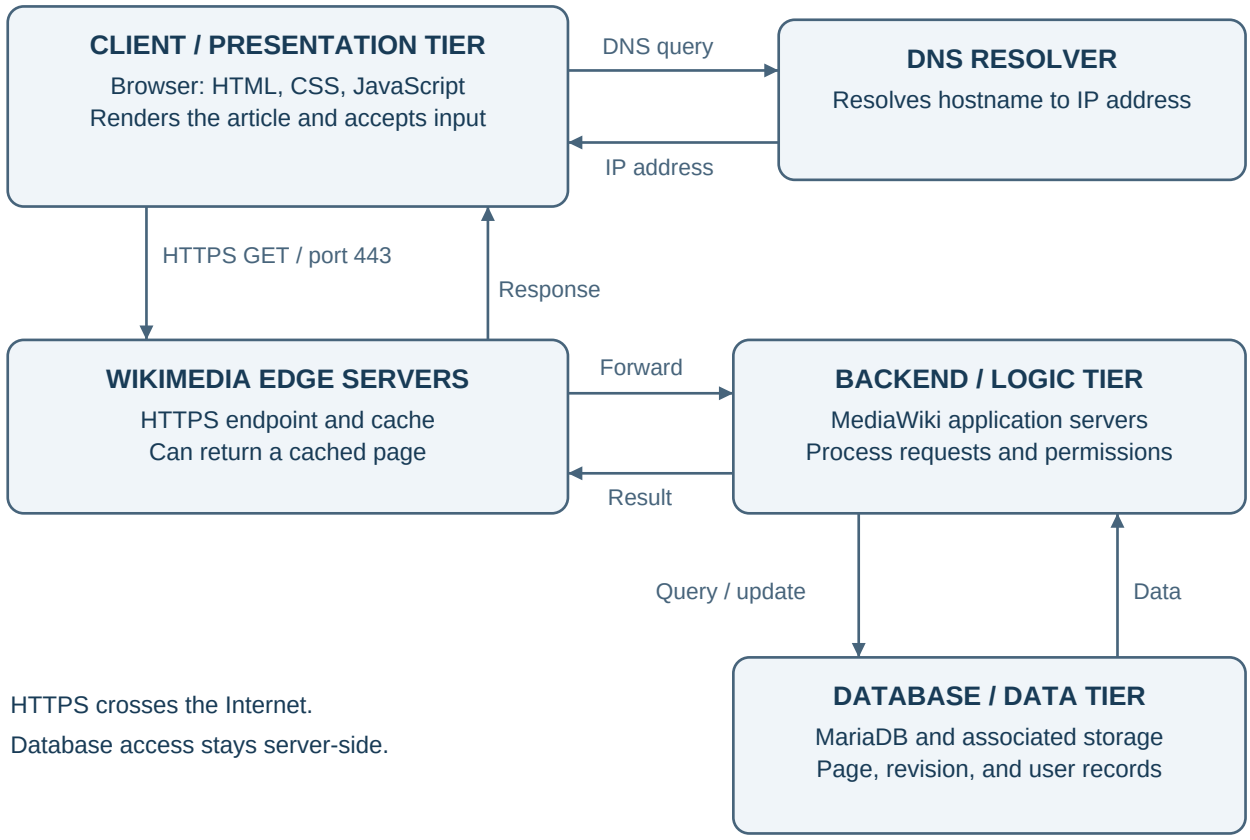
6. Render the page. The browser reads the HTML and builds the page structure. It requests additional resources such as CSS, scripts, and images, unless usable copies are already cached. CSS controls presentation, JavaScript adds behavior, and the browser calculates the layout and paints the page. Display can begin while other resources continue loading; one visible page commonly involves many requests. [5]

5. Frontend, backend, database, and APIs

The **frontend** is the browser-facing interface: article text, navigation, search controls, and editing screens. HTML provides structure, CSS provides styling, and JavaScript supports interaction. The **backend** runs on Wikimedia's servers. MediaWiki processes actions, applies permissions, and prepares page content. The **database** tier uses MariaDB to hold structured records such as page metadata, revision information, and user information; associated storage holds content and media. The browser does not directly query these databases. [2, 3]

This can be understood as a logical **three-tier architecture**: presentation, application logic, and data. The tiers describe responsibilities, not an exact count of machines. An **API** provides a defined way for software to request data or actions from the backend. For example, MediaWiki's APIs can return structured page information for client code, instead of a complete HTML page. The backend still controls access to the data. Figure 1 shows the main roles and the return path of a response. [2]

Figure 1. Wikipedia client-server architecture



Original simplified diagram based on [2] and [3]. The backend and database path is used when needed; caches may satisfy a request earlier. Internal arrows show logical communication, not browser HTTPS connections.

References

[1] MDN Web Docs. How the web works.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works

[2] MediaWiki. Manual: MediaWiki architecture.
https://www.mediawiki.org/wiki/Manual:MediaWiki_architecture

[3] Wikitech. MediaWiki at WMF.
https://wikitech.wikimedia.org/wiki/MediaWiki_at_WMF

[4] MDN Web Docs. Overview of HTTP.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

[5] MDN Web Docs. Populating the page: how browsers work.
https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/How_browsers_work